

DEPLOYMENT & MODEL LOADING

Continuum OS: Software Deployment & LLM Loading

How the built Continuum OS software actually reaches Raspberry Pi 5 hardware, and the real, tested mechanism by which model weights are demand-paged into a running cluster.

Doc	ARCH-004
Status	Grounded in the built codebase at ~/continuum-os
Date	2026-09-19
Companion to	ARCH-001 (architecture), ARCH-002 (hardware/build), ARCH-003 (runtime diagram)

Contents

- 00** Scope — what this document describes
- 01** From source tree to a bootable node image
- 02** Node provisioning — from generic image to a running role
- 03** continuumd's own boot sequence (PID 1)
- 04** The weight file — how a model is packaged
- 05** Loading a model into the cluster, step by step
- 06** Tiered page resolution and what's simulated vs. real
- 07** Honest status — what's verified, what isn't

00 Scope — what this document describes

ARCH-002 specified a hardware/build plan; this document describes what actually happened when that plan was built. Every path, port number, struct field, and function name below is copied from the real source tree at `~/continuum-os`, not re-derived from the architecture docs — where the build simplified, skipped, or simulated something the architecture called for, that gap is called out explicitly in §07 rather than papered over.

TWO QUESTIONS, TWO HALVES

§§01–03 answer "how does the software get onto the hardware" — from a monorepo checkout to a running `continuumd` supervising services on a Pi 5. §§04–06 answer "how does a model get into the cluster" — from a weight file on disk to resident, dequantized tensors inside a running `shard-execd` process, via `swapd`'s demand-paging mechanism.

01 From source tree to a bootable node image

Continuum ships as one Buildroot-based node image, assembled in two stages: this repo's own CMake build compiles every daemon, then Buildroot's `post-build.sh` hook stages those binaries into the target root filesystem. The image is role-agnostic — the same image boots a control-plane node or a compute node; role is decided later, in §02.

- 01 Cross-compile every daemon first: `cmake --preset aarch64-musl -S . -B build-aarch64` then `cmake --build build-aarch64 --target continuumd node-agentd swapd shard-execd schedulerd api-gatewayd page-directoryd membershipd consold`
- 02 Point Buildroot at that output: `CONTINUUM_BUILD_DIR=$(pwd)/build-aarch64` make, using `image/buildroot/continuum_defconfig`
- 03 Buildroot builds the kernel (with `image/kernel/continuum-fragment.config`'s `io_uring/userfaultfd/cgroup-v2` requirements) and the base rootfs, then invokes `post-build.sh $TARGET_DIR`
- 04 `post-build.sh` copies `build-aarch64/kernel/continuumd` to `${TARGET_DIR}/init` — not a file under `/opt`, but literally `/init`, because `BR2_INIT_NONE=y` in the `defconfig` hands PID 1 straight to it
- 05 `post-build.sh` copies each of the eight service binaries into `${TARGET_DIR}/opt/continuum/bin/`, and creates empty `/run/continuum` and `/var/lib/continuum`
- 06 `post-build.sh` copies the config `*templates*` (`image/buildroot/config-templates/`) into `${TARGET_DIR}/etc/continuum/` — deliberately the generic templates, not a filled-in config, so one image build is never accidentally specific to one node
- 07 Buildroot packages the result into a single flashable image

Fig. 1 — `image/buildroot/post-build.sh`'s actual staging sequence (services/, `kernel/src/main.c`, `image/buildroot/post-build.sh`).*

WHY CONTINUUMD IS `/INIT`, NOT A SYSTEMD UNIT

`BR2_INIT_NONE=y` in `continuum_defconfig` tells Buildroot not to install any init system at all. The kernel's own `init=` boot argument points at `/init` directly, and that file is `continuumd` itself. There is no `systemd`, no shell-driven `init`, and no package manager anywhere in the image — `kernel/src/main.c`'s own header comment calls this out: `continuumd` "is PID 1 on every node", the thing that makes the distributed-OS framing literal rather than aspirational at this hardware tier.

The resulting rootfs layout, as staged by `post-build.sh`:

```
/init          continuumd (PID 1)
/opt/continuum/bin/
node-agentd swapd shard-execd schedulerd
api-gatewayd page-directoryd membershipd consoled
/etc/continuum/    config templates (role-specific set copied in at $02)
/run/continuum/    created empty; sockets/pidfiles land here at runtime
/var/lib/continuum/ created empty; per-node persistent state
```

02 Node provisioning — from generic image to a running role

The image from §01 is identical for every board; a node's role — control-plane or compute — and its identity (node ID, peer addresses, ports, file paths) are filled in afterward, per `image/buildroot/config-templates/README.md`'s own instructions: copy the config set matching a node's role into `/etc/continuum/`, then fill in `node_id`, peer IPs, and paths before first boot.

Which services a role runs

Each role's `continuumd.conf` names the exact children that role's `continuumd` supervises — read directly from the checked-in templates:

Role	continuumd supervises	Reference IP (6-node cluster)
control-plane (×3)	page-directoryd, membershipd, schedulerd, api-gatewayd, consoled	10.0.0.11 / .12 / .13
compute (×3)	swapd, node-agentd, shard-execd	10.0.0.21 / .22 / .23

Ports and sockets, as provisioned

Every value below is copied from the checked-in `*.conf` templates, not invented for this document:

Service	Listen	Purpose
api-gatewayd	TCP :8080	OpenAI-compatible HTTP surface
schedulerd	unix:/run/continuum/schedulerd.sock	receives dispatch requests from api-gatewayd; routes by model name, e.g. <code>route toy-model 10.0.0.21:7301</code>
page-directoryd	Raft :7101, client :7111	3-way Raft quorum across control-plane peers (peer 1/2/3 10.0.0.1{1,2,3} 7101)
membershipd	Raft :7102, client :7112	a <i>*separate*</i> Raft group from page-directoryd, on purpose — a directory snapshot/compaction pause must never risk quorum membership stability
consoled	HTTP :7500, telemetry ingest :7401	operator console + node telemetry sink
swapd	unix:/run/continuum/swapd.sock	demand-paging service; also opens a shared <code>/dev/shm</code> DRAM pool and page table (§05)
node-agentd	TCP :7201	remote page fetch — how one compute node answers another's cache miss
shard-execd	TCP :7301	prompt dispatch / token streaming — deliberately TCP, not a Unix socket, since this is the one hop that crosses physical boards (see §07, bug #5)

CONFIG IS NEVER BAKED INTO THE IMAGE

`post-build.sh`'s own comment is explicit about this: node-specific config "is NOT baked into the image — it's provisioned per node at flash/first-boot time." That's what keeps one Buildroot output usable for all six boards in the reference cluster, and what makes re-provisioning a replacement board a config copy, not a rebuild.

03 continuumd's own boot sequence (PID 1)

Once a node's kernel hands off to `/init`, `kernel/src/main.c` runs the same sequence on every node regardless of role — only the config file's service list differs.

- 01 Parse `-config /etc/continuum/continuumd.conf` (or `-v` for debug logging); exit if it fails to load
- 02 Open the boot-processor UART heartbeat link (`cfg.uart_device`) at 115200 8N1 — if the device isn't present, log a warning and continue with no hardware heartbeat rather than failing, so the same binary runs on a dev machine with nothing plugged in

- 03 Block SIGCHLD/SIGTERM/SIGINT and pick them up via a `signalfd`, rather than a signal handler
- 04 Arm two `timerfds`: a 250ms supervision tick, and (only if the UART opened) a `heartbeat_interval_ms-period` timer, default 2000ms
- 05 `supervisor_init()` + `supervisor_start_all()` — fork/exec every service named in this node's config as a separate, fault-isolated child process
- 06 Enter the `epoll` loop: on SIGCHLD, reap and let the supervisor decide whether to restart; on the 250ms tick, run supervision bookkeeping; on the heartbeat timer, write one 'H' byte to the boot-processor UART
- 07 On SIGTERM/SIGINT: signal every child, wait up to 3s for a clean exit, then return

Fig. 2 — *continuumd's actual startup, from kernel/src/main.c.*

`continuumd`'s own module comment is candid about one simplification worth repeating here: this reference build runs `continuumd` as an ordinary process rather than literally as kernel-assigned PID 1, so it can be developed and tested on an ordinary Ubuntu machine. The supervision, `cgroup`, and heartbeat logic is identical either way — only "mount /proc myself because I am PID 1" is skipped outside the real Buildroot image.

04 The weight file — how a model is packaged

ARCH-002 named a `safetensors`-style format as the eventual target. The actual build uses a much smaller custom format instead — `lib/toymodel/include/toymodel_format.h`'s own comment explains why: a full `safetensors` reader is a lot of parsing machinery in service of proving one specific mechanism (demand-paging tensor bytes through `swapd`'s tiered resolution), and that mechanism doesn't care what the header parser looks like — only that every tensor lands on a page boundary, which this format guarantees too.

Layout

A `toymodel_header_t` (magic `CTMDLv1`, page size, per-tensor scale, and one {`start_page`, `n_pages`} location per tensor) occupies page 0 in its entirety — read directly off disk by whoever loads it, since it's metadata, not part of the demand-paged region. Every tensor after it starts on its own page boundary, and each tensor's `start_page` is the **same** absolute page index that `swapd`'s paging engine uses for its `pread()`s — no translation table between "which page holds tensor X" and "fetch page N" is needed anywhere in the system.

`tools/gen_toy_weights.c` writes this file for the model actually used in end-to-end testing — byte-level vocabulary (256 tokens, so no separate tokenizer), one attention block, one FFN block, int8 weights with a single shared dequantization scale of 0.05. Running it prints the exact page layout it wrote:

Tensor	Shape	Bytes	Pages	Page range
header	—	4,096 (page 0, metadata only)	1	0
embedding	[256, 32]	8,192	2	1–2
w_q	[32, 32]	1,024	1	3
w_k	[32, 32]	1,024	1	4
w_v	[32, 32]	1,024	1	5
w_o	[32, 32]	1,024	1	6
w_ff1	[32, 64]	2,048	1	7
w_ff2	[64, 32]	2,048	1	8
unembedding	[32, 256]	8,192	2	9–10

Total: 11 pages × 4,096-byte page size = 45,056 bytes (44 KB) at 4-byte-per-dim/32-dim/64-ff_dim — trivially small on purpose, since proving the paging mechanism needs a model that's real, not one that's large. `gen_toy_weights` prints the exact `tensor_region_bytes` value a node's `swapd.conf` must be given to match, so the two never drift apart by hand-editing.

05 Loading a model into the cluster, step by step

This is the actual, tested path — `gen_toy_weights` → a compute node's `swapd` → `shard-execd` — as implemented in `services/swapd/src/paging_engine.c`, `lib/pageclient/src/pageclient.c`, and `services/shard-execd/src/toy_model.c`.

- 01 An operator (or provisioning script) runs `gen_toy_weights <path> [seed]` on the compute node, producing a page-aligned weight file at the path `swapd.conf`'s `weight_file` will name
- 02 That node's `swapd.conf` sets `weight_file`, `tensor_region_bytes` (matching `gen_toy_weights`' printed total), `page_size` 4096, and this node's `node_id`
- 03 `swapd` starts under `continuumd`, opens the weight file read-only, and `mmap`'s an anonymous region of `tensor_region_bytes` — this region, not the file itself, is what gets demand-paged
- 04 `swapd` registers that region with `userfaultfd` in MISSING mode (`try_setup_real_uffd`); it also creates the node's shared page table and DRAM pool under `/continuum-pt-<node_id>` and `/continuum-pool-<node_id>` (POSIX shared memory, `lib/pagetable`)
- 05 `shard-execd` starts, and `pageclient_connect`() attaches to that same shared page table and pool — retrying for a few seconds if `swapd` hasn't created them yet
- 06 `toy_model_load`() reads the header directly from the file (page 0 — metadata, never demand-paged) to learn every tensor's `{start_page, n_pages}`
- 07 For every page of every tensor, `shard-execd` calls `pageclient_get_page(pc, page_index)`. A hit in the

- shared page table returns a pointer straight into the DRAM pool — zero-copy, no IPC
- 08 On a miss, pageclient sends a `WIRE_PAGE_FETCH_REQ` to swapd over `/run/continuum/swapd.sock` — the identical message a `*remote*` node's swapd would send on a cross-node miss
 - 09 swapd's `paging_engine_ensure_resident()` touches one byte of that page inside the uffd-registered region; the kernel blocks the access and delivers a fault event to swapd's fault thread
 - 10 The fault thread resolves the page's content: `pread()` from the local weight file at `page_index * page_size` first; only on a short/failed local read does it try each configured remote peer's swapd over TCP with the same `WIRE_PAGE_FETCH_REQ/RESP` pair
 - 11 swapd answers the kernel fault with `UFFDIO_COPY`, copies the page into this node's shared DRAM pool, and records it in the shared page table at `WIRE_TIER_LOCAL_DRAM` with a last-touched timestamp for later LRU eviction
 - 12 shard-execd's `pageclient_get_page()` call returns; `toy_model_load()` copies the raw int8 bytes out, multiplies every element by the header's shared `scale` (0.05), and moves to the next tensor
 - 13 Once all eight tensors are loaded, `toy_model_load()` logs the total page count fetched and returns a ready `toy_model_t` — shard-execd can now run `toy_model_forward_step()` entirely against already-resident float arrays in its own process memory

Fig. 3 — one model load, from weight file on disk to resident float tensors inside shard-execd.

PLACEMENT IS A STATIC ROUTE, NOT AUTOMATIC SCHEDULING

Which compute node loads which model is decided by a line in that node's control-plane peer's `schedulerd.conf` — e.g. `route toy-model 10.0.0.21:7301` — not by any automatic placement logic. `api-gatewayd` forwards a request to `schedulerd` over its Unix socket; `schedulerd` looks the model name up in that static route table and dispatches `WIRE_DISPATCH_PROMPT` to the matching `shard-execd` over TCP. Multi-shard pipeline placement across several compute nodes is the architecture's stated direction but isn't what this build's route table expresses today — it's a single-hop route per model name.

06 Tiered page resolution and what's simulated vs. real

`lib/wire/include/wire.h` defines four resolution tiers — `WIRE_TIER_LOCAL_DRAM` (1), `WIRE_TIER_REMOTE_DRAM` (2), `WIRE_TIER_LOCAL_NVME` (3), `WIRE_TIER_REMOTE_NVME` (4) — matching ARCH-001's tiering model. The actual resolution code in `paging_engine.c` is more direct than four separate code paths: `resolve_page_content()` tries the local weight file with one `pread()`, and on failure tries each configured remote peer with one uniform `PAGE_FETCH_REQ/RESP` exchange. The module's own comment explains the collapse: from the requester's side, "remote DRAM" and "remote NVMe" both just mean "ask a specific peer for this page" — the tier distinction matters for **how a peer answers**, not for how a page is requested, so this reference build doesn't duplicate that logic four ways.

Tier	What ARCH-001 specifies	What this build does
Local DRAM	Page already resident in this node's DRAM pool	Exactly this — <code>shm_pagetable_get()</code> hit, zero-copy read (real)
Remote DRAM / Remote NVMe	Ask a peer node; peer answers from its own DRAM or NVMe depending on its own tier state	Folded into one path: <code>WIRE_PAGE_FETCH_REQ</code> to each configured peer in order, first usable <code>WIRE_PAGE_FETCH_RESP</code> wins (real, but tier-agnostic from the requester's side)
Local NVMe	Async read via <code>io_uring</code>	A synchronous <code>pread()</code> against the local weight file — the demand-paging <code>*mechanism*</code> (<code>userfaultfd</code>) is real; <code>io_uring</code> is named in ARCH-002 but not implemented, a swappable I/O-backend choice that doesn't touch the fault-handling path
<code>userfaultfd</code> itself	Real kernel demand paging	Real when available (<code>UFFDIO_REGISTER</code> succeeds) — falls back to an explicit page-touch-and-copy simulation (<code>paging_engine_ensure_resident</code> 's non-uffd branch) when <code>userfaultfd()</code> is unprivileged or unavailable, so the same binary still runs correctly on a dev machine without <code>CAP_SYS_PTRACE</code> -adjacent permissions

07 Honest status — what's verified, what isn't

Carried over directly from the project's own README, since a deployment document that omits it would be misleading:

- The full CMake build compiles cleanly and 20 `ctest` tests pass, including multi-process end-to-end tests for paging, Raft, and console telemetry — on native x86_64/glibc. Cross-compiling to `aarch64-linux-musl` (the actual Pi 5 target) is written into the CMake presets but has not been exercised in this environment.
- The Buildroot image itself has never been run — no Buildroot checkout was attempted (a full build is hours and gigabytes, unlike the firmware below). `image/` holds real, complete config: `defconfig`, kernel fragment, post-build script, per-role templates — just never exercised end to end.
- The boot-processor firmware, by contrast, was actually built against a real downloaded ARM toolchain and Pico SDK: zero compiler warnings, a byte-correct `.uf2` (verified UF2 magic numbers, ELF32/ARM header, entry point in the RP2040's XIP

region). What's unverified is that firmware running on real Pico hardware — none was available to flash.

- `io_uring` is a synchronous `pread()` for now (§06) — a named, deliberate simplification, not an oversight.
- No durable Raft persistence — `lib/raftnet` keeps its log purely in-memory, correct for consensus but a full quorum restart loses history.
- The KV cache lives in `shard-execd`'s own process memory, not routed through `swapd`'s paged tensor region — `lib/kvcache`'s prefix-trie is wired in for prefix dedup, but not for the tier-aware KV swapping ARCH-001 describes.

WHY THIS LEVEL OF CANDOR

Every gap above was found by actually building and running the system — not by inspection. The Raft split-vote livelock, the blocking-timerfd hang, the `jsmn` double-count bug, and the Unix-socket-instead-of-TCP placement bug (this document's §02 already reflects the fix) were each caught by a specific end-to-end test failing. That's the argument for treating this document, and the diagram it accompanies, as a description of a system that has actually run — not a restatement of the plan that preceded it.